códigofacilito

Bootcamp · Módulo 5: Edge, Storage y serverless

Eventos y Broadcast

Clase 6 de 6 · Supabase + Next.js



Rodrigo Méndez
rodrigomendez.dev

códigofacilito

Tabla de contenido

/01 El problema y la meta

/02 Arquitectura basada en eventos

/03 Broadcast

/04 Binario y escala

/05 Para cerrar el curso



códigofacilito

/01

El problema y la meta

Qué vamos a construir hoy, y por qué.



códigofacilito

EL PROBLEMA

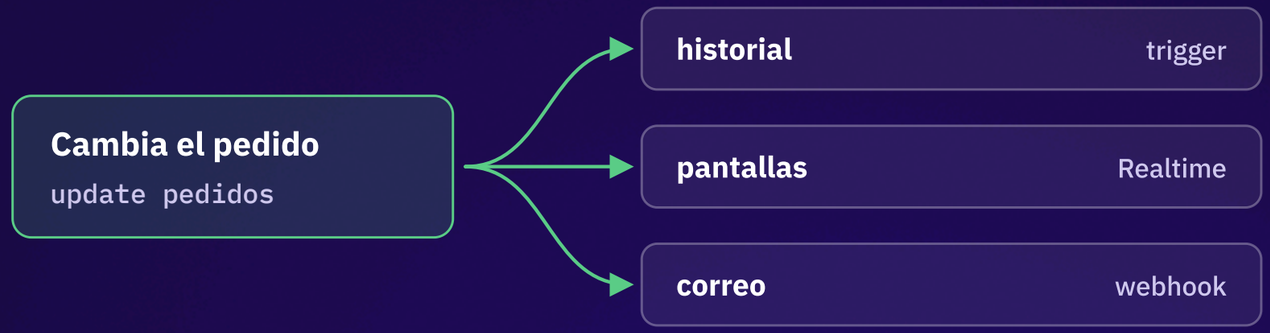
Cuando rastreas un paquete ves cada paso con su hora. ¿Cómo se registra sin que nadie lo anote a mano?

EN CADENA: EL BOTÓN HACE TODO



Si una parte falla, falla todo

CON EVENTOS: UN CAMBIO, VARIAS REACCIONES

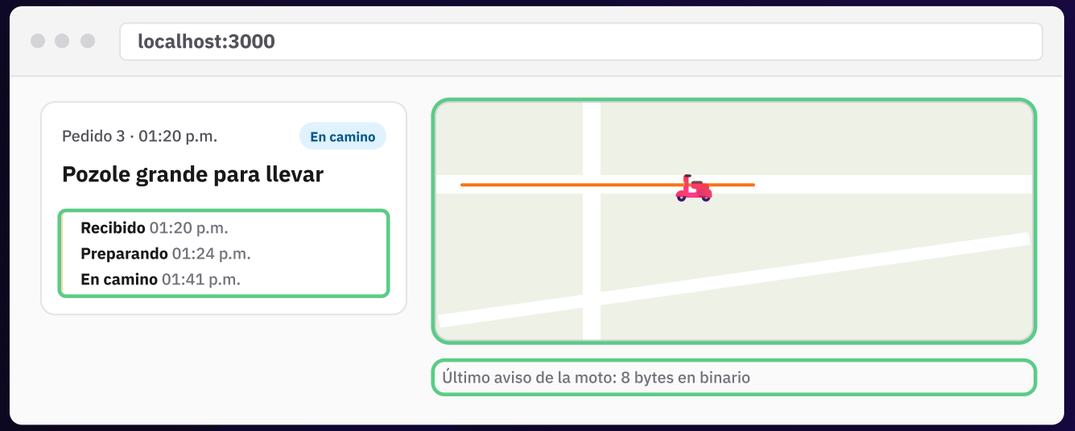


Cada reacción es independiente

códigofacilito

LA META DE HOY

El historial y la moto en el mapa



1. **El historial se llena solo**
   Cada cambio de estado con su hora.
2. **La moto se mueve en el mapa**
   Una posición por segundo, en vivo.
3. **Cada aviso pesa 8 bytes**
   Empacado en binario.

RASTRO, SIGUE TU PEDIDO · UNA SOLA APP QUE CRECE CLASE A CLASE

CLASE 1	CLASE 2	CLASE 3	CLASE 4	CLASE 5	CLASE 6
Base de datos	Auth y RLS	Realtime	Edge Functions	Storage	Eventos y Broadcast
					Hoy

códigofacilito

/02

Arquitectura basada en eventos

Cuando pasa algo, otras partes reaccionan solas.



códigofacilito

UN EVENTO ES ALGO QUE PASÓ

Ya lo hiciste sin saber

CLASE 2

Te registras y nace tu perfil

Un trigger en `auth.users` crea la fila en `perfiles`.

CLASE 3

Cambia un pedido y la pantalla avanza

Realtime avisa a los navegadores suscritos.

CLASE 4

Sale el pedido y llega el correo

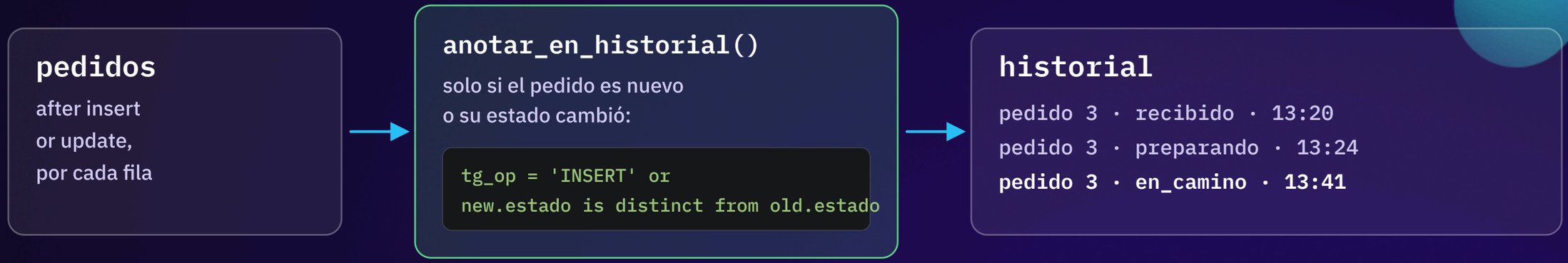
Un webhook llama a la Edge Function.

> En los tres, quien causó el cambio no sabe quién reacciona: eso es una arquitectura basada en eventos.

códigofacilito

TRIGGER

El historial se llena solo



La página trae cada pedido con su historial en una sola consulta; el de la clase 3 lo muestra en vivo.

- RLS: ves el historial si puedes ver el pedido
- `select("*, historial(...)")` trae la relación

códigofacilito

/03 Broadcast

Mensajes al momento, sin pasar por la base.



códigofacilito

DOS FORMAS DE REALTIME

Postgres Changes o Broadcast

POSTGRES CHANGES · CLASE 3

Cambios de la base

Primero se guarda la fila y luego se avisa. Sirve para lo que tiene que quedar registrado: el estado de un pedido.

BROADCAST · HOY

Mensajes entre clientes

Van por un canal y no se guardan. Sirve para lo que solo importa en el momento: la ubicación de cada segundo.

> Guardar 60 posiciones por minuto por cada moto llenaría la base de filas que nadie vuelve a leer.

códigofacilito

UN CANAL POR PEDIDO

Cómo viaja la posición de la moto

La tienda · `Simulador`

recorre la ruta grabada,
un punto por segundo

```
send({ event: "posicion", … })
```

→

Canal `pedido-3`

Realtime lo reparte a
los suscritos; no guarda nada

→

El cliente · `SeguirMoto`

escucha el evento y
mueve la moto en el mapa

```
on("broadcast", { event: "posicion" })
```

`canal público` En el curso cualquiera que sepa el nombre del canal puede escucharlo. En producción, la ubicación va en **canales privados**, con políticas (Realtime Authorization).

códigofacilito

/04 Binario y escala

Menos bytes por mensaje y menos mensajes.



códigofacilito

PAYLOADS BINARIOS

La misma posición, en texto o en binario

TEXTO · JSON

`{"lat":19.4185,"lng":-99.1592}`

unos 30 a 32 bytes

BINARIO · `Float32Array`

`2 números × 4 bytes`

8 bytes

> Cada número ocupa siempre 4 bytes, sin nombres de campos ni comillas. Y se manda el `.buffer`, no el arreglo.

códigofacilito

LA CUENTA EN LA PIZARRA

¿Y si fueran 200 repartidores?

200 motos × 1 aviso por segundo	200 mensajes/s
Límite del plan gratis de Supabase Realtime	100 mensajes/s
Juntar: un solo mensaje por segundo con las 200 posiciones	1 mensaje/s
Empacar en binario: 200 × 8 bytes	1,600 bytes

> Para escalar: juntar los avisos en menos mensajes y empacarlos en binario. `empacar()` ya recibe varias posiciones.

códigofacilito

EL ERROR DEL DÍA

El binario no llega, o llega como texto

LIBRERÍA VIEJA

No llega, sin error

Con `@supabase/supabase-js` 2.90.0 el texto llega y el binario se descarta en silencio. El proyecto usa una versión nueva.

SIN `.buffer`

Llega como texto

Si mandas el `Float32Array`, llega convertido: `{"0":19.4115…,"1":-99.1686}`.

> Si el binario no se comporta, revisa la versión de la librería y que mandes un `ArrayBuffer`.

códigofacilito

MÁS ALLÁ DE LA MOTO

Qué sí y qué no va en tiempo real

SÍ: LO QUE IMPORTA EN EL MOMENTO

Broadcast y Presence

Cursores en vivo, el "escribiendo...", votaciones en vivo, juegos por turnos, ubicaciones.

NO: LO QUE NO SE PUEDE PERDER

A la base de datos

Pagos, pedidos y cualquier cosa que tenga que quedar registrada: se guarda primero y luego se avisa.

> Un mensaje de Broadcast que nadie escuchó se perdió. Si no se puede perder, va a la base.

códigofacilito

/05

Para cerrar el curso

Lo que te llevas, la tarea y lo que sigue.



códigofacilito

LO QUE TE LLEVAS HOY

Tres ideas

1

Un evento, varias reacciones

Trigger, Realtime y webhook reaccionan sin que el origen los conozca.

2

Broadcast no guarda nada

Mensajes al momento para lo que solo importa en el momento.

3

Binario: el mismo dato con menos bytes

Para escalar, juntar los avisos y empacarlos.

códigofacilito

TAREA

Construye tú solo lo que viste hoy

CÓMO SÉ QUE YA QUEDÓ

- [ ] Cada cambio de estado aparece en el historial, con su hora.
- [ ] El cliente ve el historial actualizarse sin recargar.
- [ ] La moto se mueve en el mapa del cliente.
- [ ] Con "Mandar en binario", cada aviso pesa 8 bytes.

¿NO TE SALIÓ?

No pasa nada

Compara con `clase-6-final`: es el proyecto terminado.

códigofacilito

PROYECTO FINAL SUGERIDO

Adapta Rastro a un negocio real

- **Florería**
- **Farmacia**
- **Lavandería**
- **Papelería**

> Mismas piezas: tablas con RLS, sesión, Realtime, una función, Storage y eventos. Otro negocio, otras reglas.

Recursos para profundizar

- Broadcast: supabase.com/docs/guides/realtime/broadcast
- Canales privados (Realtime Authorization): supabase.com/docs/guides/realtime/authorization
- Triggers en Postgres: supabase.com/docs/guides/database/postgres/triggers
- Límites de Realtime: supabase.com/docs/guides/realtime/limits



códigofacilito

CIERRE DEL CURSO

Seis clases, una app completa

CLASE 1	CLASE 2	CLASE 3	CLASE 4	CLASE 5	CLASE 6
Base de datos	Auth y RLS	Realtime	Edge Functions	Storage	Eventos y Broadcast

Gracias. Nos vemos en el proyecto final.

rodrigomendez.dev · github.com/rodrigo2392